Example 1:

PROBLEM 3. (14 pts) Give the output for the following piece of code, assuming that it was compiled on an IA-32 machine. Variable i, j, and k have memory addresses 200, 300 and 400, respectively.

```
#include <stdio.h>
int main() {
      int i = 4; // Assume that i is stored at memory address 200
      int *j = &i; // Assume that j is stored at memory address 300
      int *k = (int *) i; // Assume that k is stored at memory address 400

      printf("%d,%d,%d \n", (int) i, (int) &i, (int) (i+1));
      printf("%d,%d,%d \n", (int) j, (int) &j, (int) (j+1));
      printf("%d,%d,%d \n", (int) k, (int) &k, (int) (k+1));
      return 0;
}
```

This program prints out three lines. Each line has three values that are separated by a comma. What is the output?

Example 1:

PROBLEM 3. (14 pts) Give the output for the following piece of code, assuming that it was compiled on an IA-32 machine. Variable i, j, and k have memory addresses 200, 300 and 400, respectively.

```
#include <stdio.h>
int main() {
      int i = 4; // Assume that i is stored at memory address 200
      int *j = &i; // Assume that j is stored at memory address 300
      int *k = (int *) i; // Assume that k is stored at memory address 400

      printf("%d,%d,%d \n", (int) i, (int) &i, (int) (i+1));
      printf("%d,%d,%d \n", (int) j, (int) &j, (int) (j+1));
      printf("%d,%d,%d \n", (int) k, (int) &k, (int) (k+1));
      return 0;
}
```

This program prints out three lines. Each line has three values that are separated by a comma. What is the output?

4, 200, 5
200, 300, 204
4, 400, 8

Example 2:

PROBLEM 5. (8+4=12 pts) Consider the following functions:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

The following three code fragments call these functions:

A
```
for (i = min(x, y); i < max(x, y); incr(&i, 1))  t += square(i);
```

B
```
for (i = max(x, y)-1; i >= min(x, y); incr(&i, -1))  t += square(i);
```

a) Assume x=10 and y=100. Fill in the following table indicating the number of times each of the four functions is called in code fragments **A** and **B**:

Code	min	max	incr	square
A				
B				

Example 2:

PROBLEM 5. (8+4=12 pts) Consider the following functions:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

The following three code fragments call these functions:

A
```
for (i = min(x, y); i < max(x, y); incr(&i, 1))  t += square(i);
```

B
```
for (i = max(x, y)-1; i >= min(x, y); incr(&i, -1))  t += square(i);
```

a) Assume x=10 and y=100. Fill in the following table indicating the number of times each of the four functions is called in code fragments **A** and **B**:

Code	min	max	incr	square
A	1	91	90	90
B	91	1	90	90

Example 2:

PROBLEM 5. (8+4=12 pts) Consider the following functions:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

The following three code fragments call these functions:

A
```
for (i = min(x, y); i < max(x, y); incr(&i, 1))  t += square(i);
```

B
```
for (i = max(x, y)-1; i >= min(x, y); incr(&i, -1))  t += square(i);
```

b) Optimize the code fragment **A**, such that the number of calls for the functions in the new code are given in the following table:

Code	min	max	incr	square
C	1	1	90	90

Example 2:

PROBLEM 5. (8+4=12 pts) Consider the following functions:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

The following three code fragments call these functions:

A
```
for (i = min(x, y); i < max(x, y); incr(&i, 1))  t += square(i);
```

B
```
for (i = max(x, y)-1; i >= min(x, y); incr(&i, -1))  t += square(i);
```

b) Optimize the code fragment **A**, such that the number of calls for the functions in the new code are given in the following table:

Code	min	max	incr	square
C	1	1	90	90

```
long low = min(x, y);
long high = max(x, y);
for (i = low; i < high; incr(&i, 1))
        t += square(i);
```

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

a) The box below shows the format of a physical address. Indicate (by labeling the diagram) the fields that would be used to determine the following:

BO The block offset within the cache line
SI The set index
T The cache tag

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

a) The box below shows the format of a physical address. Indicate (by labeling the diagram) the fields that would be used to determine the following:

BO The block offset within the cache line
SI The set index
T The cache tag

13	12	11	10	9	8	7	6	5	4	3	2	1	0
Tag	Tag	Tag	Tag	Tag	Tag	Tag	Tag	Tag	Tag	SI	BO	BO	BO

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

b) Given that the address of `arr[0][0]` has value 0x2CCC, perform a cache address translation to determine the block offset and set index for the first item in the array.

SI = 0x_____
BO = 0x_____

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

b) Given that the address of `arr[0][0]` has value 0x2CCC, perform a cache address translation to determine the block offset and set index for the first item in the array.

SI = 0x___1____
BO = 0x___4____

13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	1	0	0	1	1	0	0

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

c) For each element in the matrix `int arr[4][3]`, label the diagram below with the set index that it will map to.

arr	0	1	2
0			
1			
2			
3			

Example 3:

PROBLEM 7. (3+2+6+11=22 pts) For this problem assume the following:

- Memory is byte addressable.
- Physical addresses are 14 bits wide.
- The cache is 2-way set associative with an 8 byte block-size and 2 sets.
- Least-Recently-Used (LRU) replacement policy is used.
- `sizeof(int)` = 4 bytes.

Consider a matrix declared as `int arr[4][3]`. Assume that the array has already been initialized.

c) For each element in the matrix `int arr[4][3]`, label the diagram below with the set index that it will map to.

arr	0	1	2
0	1	0	0
1	1	1	0
2	0	1	1
3	0	0	1

Example 3:

d) Assume the cache stores only the matrix elements; variables i, j, and sum are stored in registers.

```
int i, j;
int sum = 0;
for(i=0; i<4; i++){
        for(j=0; j<3; j++){
              sum += arr[i][j];
        }
}
/* second access begins */
for(i=2; i>=0; i=i-2){
        for(j=0; j<3; j++){
              sum += arr[i][j];
              sum += arr[i+1][j];
        }
}
/* second access ends */
```

arr	0	1	2
0			
1			
2	**h**		
3			

Assume the above piece of code is executed. Fill out the table to indicate if the corresponding memory access will be a hit (**h**) or a miss (**m**) when accessing the array `arr[4][3]` for the **second** time (between the comments 'second access begins' and 'second access ends').

Example 3:

d) Assume the cache stores only the matrix elements; variables i, j, and sum are stored in registers.

```
int i, j;
int sum = 0;
for(i=0; i<4; i++){
        for(j=0; j<3; j++){
              sum += arr[i][j];
        }
}
/* second access begins */
for(i=2; i>=0; i=i-2){
        for(j=0; j<3; j++){
              sum += arr[i][j];
              sum += arr[i+1][j];
        }
}
/* second access ends */
```

arr	0	1	2
0	m	m	h
1	m	h	h
2	**h**	h	h
3	h	h	h

Assume the above piece of code is executed. Fill out the table to indicate if the corresponding memory access will be a hit (**h**) or a miss (**m**) when accessing the array `arr[4][3]` for the **second** time (between the comments 'second access begins' and 'second access ends').